

Plan for Full-Ring Ultrasound Speed-of-Sound Reconstruction Using a Tensorized Fourier Neural Operator

September 2026

Abstract

This document describes a staged plan for training a Tensorized Fourier Neural Operator (TFNO) to estimate a two-dimensional speed-of-sound field from a complete circular ultrasound acquisition. Each example contains a known transmit pulse, measurements from 256 transmitters and 256 receivers, and a 512×512 target speed-of-sound image. The plan covers acquisition geometry, full-wave simulation, storage, signal representation, network architecture, training, validation, and software organization. Only the recommended full-ring acquisition is considered.

1 Objective and Scope

The objective is to learn the nonlinear inverse operator

$$\mathcal{G}^{-1} : \{s(t), d(\theta_{\text{tx}}, \theta_{\text{rx}}, t)\} \mapsto c(x, y), \quad (1)$$

where $s(t)$ is the transmitted pulse, d is the measured pressure, and $c(x, y)$ is the spatial speed of sound. The acquisition contains 256 transmit events, with all 256 elements recording each event. Therefore, one unprocessed measurement has the form

$$d \in \mathbb{R}^{256 \times 256 \times T}, \quad (2)$$

and the reconstruction target is

$$c \in \mathbb{R}^{512 \times 512}. \quad (3)$$

The proposed method follows the neural-operator formulation demonstrated for ultrasound tomography by Dai et al. [1], while extending it from a time-of-flight field to multi-frequency full-wave information and from 128 to 256 ring elements. Fourier neural operators learn maps between functions by parameterizing integral kernels in Fourier space [2, 3]. Tensorization reduces the number of spectral parameters and is available in the official NeuralOperator implementation [4].

2 Fixed Acquisition Geometry

All training, validation, test, and inference examples shall use one fixed physical geometry. This is essential because changing the ring radius or grid spacing changes the inverse operator that the network must learn.

The initial geometry will use:

- a 512×512 Cartesian computational grid;

- isotropic spacing of approximately 1 mm;
- a fixed ring radius of approximately 230 mm;
- 256 equally spaced transmit-receive elements;
- water coupling outside the body;
- fixed element numbering and angular origin;
- a fixed recording interval and number of temporal samples.

Each CT-derived medium will be centered and padded into this common field of view. It may be downsampled when necessary, but it will not be independently rescaled to a different ring geometry. An image that cannot fit inside the fixed ring will be rejected or handled by a single documented preprocessing rule that is applied to the entire dataset.

3 Full-Ring Forward Acquisition

A new full-ring acquisition routine will wrap the existing `simulate_shot` function. For transmitter index i , the simulator returns

$$d_i(j, n), \quad j = 0, \dots, 255, \quad n = 0, \dots, T - 1. \quad (4)$$

Repeating this for all transmitters produces the complete tensor

$$d(i, j, n) \in \mathbb{R}^{256 \times 256 \times T}. \quad (5)$$

The acquisition implementation will:

- reset the wave solver before every transmit event;
- retain the transmitter self-channel but provide a validity mask;
- save transmitter and receiver coordinates, Δt , grid spacing, ring radius, pulse, and simulation configuration;
- support restart at the transmitter level after interruption;
- write samples incrementally rather than holding an entire dataset in memory;
- test acoustic reciprocity, $d_{ij}(t) \approx d_{ji}(t)$, as a simulation quality check.

Independent subjects or CT slices can be generated in parallel. Writes to a single sample should remain serialized to avoid corrupting the dataset.

4 Dataset Contract and Storage

Each dataset sample will contain the following arrays and metadata:

Field	Shape	Description
<code>pulse</code>	(T)	transmitted pressure waveform
<code>traces</code>	$(256, 256, T)$	full transmitter-receiver data
<code>sound_speed</code>	$(512, 512)$	target $c(x, y)$ in m/s
<code>slowness_squared</code>	$(512, 512)$	target $m = 1/c^2$
<code>ring_mask</code>	$(512, 512)$	known reconstruction support
<code>valid_mask</code>	$(256, 256)$	usable measurement pairs
<code>metadata</code>	scalar/vector	geometry, timing, and provenance

Zarr or HDF5 will be used for chunked, compressed, and resumable storage. At 512 temporal samples, one float16 trace cube requires approximately 67 MB before compression. One thousand examples therefore require approximately 67 GB for traces alone. Raw traces will be retained as archival data, while compact frequency-domain features will be used during training.

The planned dataset progression is:

1. 16 examples for end-to-end pipeline testing;
2. 200 training, 25 validation, and 25 test examples for a pilot study;
3. at least 1000 diverse training examples for the main model.

All slices from one patient must remain in the same split. The patient split will occur before slice selection or augmentation to prevent information leakage.

5 Pulse and Waveform Representation

Passing all T time samples to a two-dimensional TFNO would be expensive and would make storage and training unnecessarily difficult. Instead, the pulse and received signals will be transformed into a stabilized complex frequency response. For selected frequencies f_k ,

$$H_{ij}(f_k) = \frac{D_{ij}(f_k)S^*(f_k)}{|S(f_k)|^2 + \epsilon}, \quad (6)$$

where S and D_{ij} are the Fourier transforms of the transmitted and received signals, respectively. The regularization ϵ prevents division by low pulse energy.

The initial feature configuration will use 16 frequencies in the transmitted band, nominally 100–250 kHz. The TFNO input will include:

- real and imaginary parts of $H_{ij}(f_k)$: 32 channels;
- real and imaginary parts of the normalized pulse spectrum, broadcast over the transmitter-receiver grid: 32 channels;
- $\sin \theta_{\text{tx}}$, $\cos \theta_{\text{tx}}$, $\sin \theta_{\text{rx}}$, and $\cos \theta_{\text{rx}}$;
- normalized transmitter-receiver angular separation;
- the measurement validity mask.

The resulting initial tensor has approximately 70 channels:

$$X \in \mathbb{R}^{B \times 70 \times 256 \times 256}. \quad (7)$$

This remains a full-waveform approach because complex amplitude and phase are retained at multiple frequencies; it is not limited to a picked arrival time.

6 TFNO Reconstruction Architecture

The model will contain three modules:

1. a deterministic pulse-and-trace feature builder;
2. a TFNO backbone operating on the periodic transmitter-receiver grid;
3. a learned Cartesian decoder producing the physical image grid.

The pilot TFNO configuration will use:

- input grid: 256×256 transmitter-receiver samples;
- Fourier modes: 32×32 , with 48×48 tested later;
- hidden channels: 32, with 64 tested later;
- four Fourier layers;
- GELU activations;
- Tucker-factorized spectral weights;
- tensor rank between 0.1 and 0.2.

The TFNO produces a latent 256×256 field. A residual convolutional decoder then changes from the angular transmitter-receiver representation to the Cartesian image representation and performs learned $2\times$ upsampling to 512×512 . Keeping this decoder explicit avoids relying on library-specific super-resolution behavior.

7 Output Parameterization

The network will predict slowness-squared rather than speed directly:

$$m(x, y) = \frac{1}{c(x, y)^2}. \quad (8)$$

More specifically, it will predict contrast relative to the water background,

$$\delta m(x, y) = m(x, y) - m_{\text{water}}. \quad (9)$$

The final speed estimate is

$$\hat{c}(x, y) = \frac{1}{\sqrt{\hat{m}(x, y)}}. \quad (10)$$

This parameterization is consistent with the existing adjoint inversion, relates naturally to travel time, and allows the output to be bounded to the speed range represented in the training data. Pixels outside the known ring will be set to coupling-water speed rather than predicted.

8 Training Objective

The initial loss will combine physical-value, boundary, and spectral terms:

$$\mathcal{L} = \lambda_m \mathcal{L}_{\text{Huber}}(m, \hat{m}) + \lambda_c \|c - \hat{c}\|_1 + \lambda_g \mathcal{L}_{\text{grad}} + \lambda_f \mathcal{L}_{\text{Fourier}}. \quad (11)$$

All terms will be masked to the interior of the ring. Tissue-balanced weights will prevent the large water and soft-tissue regions from overwhelming bone, fat, and small anatomical structures. The gradient term will use Sobel derivatives, while the Fourier term will compare spatial spectra at low and mid frequencies.

Training will begin with AdamW, automatic mixed precision, batch size one or two, and gradient accumulation. Tucker factorization reduces parameter memory, but activations on a 256×256 grid will still dominate GPU use.

9 Physically Valid Augmentation

The training pipeline will use transformations that respect the ring geometry:

- joint cyclic shifts of transmitter and receiver indices with the corresponding target-image rotation;
- ring reflections with the correct element-index reversal;
- additive electronic noise;
- receiver gain variation;
- timing jitter;
- random dead channels accompanied by the validity mask;
- moderate pulse-frequency, bandwidth, phase, and amplitude variation.

Whenever the pulse is varied, the exact pulse remains an input to the model.

10 Evaluation

The test set will contain only patients absent from training and validation. Reported metrics will include:

- mean absolute error in m/s inside the ring;
- relative L_2 error;
- structural similarity index;
- spatial-gradient error;
- tissue-specific error for bone, fat, muscle, and visceral tissue;
- robustness to noise, timing jitter, gain error, and missing channels;
- inference time, parameter count, and peak GPU memory.

The principal comparison will be against the existing nonlinear adjoint FWI. A homogeneous-water reconstruction and a similarly sized U-Net will serve as additional baselines. Ablations will compare full complex frequency features, time-of-flight features, and removal of pulse conditioning.

11 Proposed Software Structure

fdtd2d/	
acquisition.py	Full 256-by-256 ring acquisition
ring_tfno/	
config.py	Geometry and model configuration
storage.py	Zarr/HDF5 sample format
features.py	Pulse/trace FFT and frequency response
dataset.py	PyTorch dataset and normalization
model.py	TFNO and Cartesian decoder
losses.py	Physical and image-space losses
metrics.py	Reconstruction metrics
checkpoint.py	Reproducible checkpoints
generate_tfno_dataset.py	Generate full-ring examples
train_tfno.py	Train and validate
evaluate_tfno.py	Held-out evaluation and comparisons
infer_tfno.py	Reconstruct one acquisition

The implementation will use PyTorch and the official `neuraloperator` package, whose TFNO implementation supports Tucker factorization and factorized spectral contractions [4]. Dependency versions and the complete training configuration will be saved with every checkpoint.

12 Implementation Sequence

1. Freeze and test the common 512×512 geometry.
2. Implement one complete 256-transmitter acquisition.
3. Verify dimensions, timing, reciprocity, and signal normalization.
4. Define resumable chunked storage and metadata validation.
5. Generate the 16-example smoke-test dataset.
6. Implement frequency-response feature extraction.
7. Implement the TFNO backbone and Cartesian decoder.
8. Overfit two examples as a model and loss correctness test.
9. Train and evaluate the 200-example pilot dataset.
10. Compare the pilot against adjoint FWI and U-Net.
11. Scale dataset generation only after the pilot demonstrates learning.

13 Acceptance Criteria

The first acquisition milestone is complete when one sample:

- contains a finite $(256, 256, T)$ trace tensor;
- uses the same geometry as every future sample;
- passes reciprocity and metadata checks;
- can be interrupted and resumed without recomputing finished shots;
- produces deterministic frequency features of the expected shape.

The pilot-model milestone is complete when the model can deliberately overfit two examples, trains stably on the pilot split, reconstructs held-out anatomy better than homogeneous water, and is evaluated using the same test cases as the adjoint FWI baseline.

References

- [1] H. Dai, M. Penwarden, R. M. Kirby, and S. Joshi, “Neural Operator Learning for Ultrasound Tomography Inversion,” *Medical Imaging with Deep Learning*, 2023. arXiv:2304.03297. <https://arxiv.org/abs/2304.03297>
- [2] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, “Fourier Neural Operator for Parametric Partial Differential Equations,” *International Conference on Learning Representations*, 2021. <https://openreview.net/forum?id=c8P9NQVtmn0>
- [3] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar, “Neural Operator: Learning Maps Between Function Spaces with Applications to PDEs,” *Journal of Machine Learning Research*, vol. 24, no. 89, pp. 1–97, 2023. <https://www.jmlr.org/papers/v24/21-1524.html>
- [4] J. Kossaifi, N. Kovachki, Z. Li, D. Pitt, M. Liu-Schiaffini, R. J. George, B. Bonev, K. Azizzadenesheli, J. Berner, and A. Anandkumar, “A Library for Learning Neural Operators,” 2024. arXiv:2412.10354. <https://arxiv.org/abs/2412.10354>
- [5] A. Tran, A. Mathews, L. Xie, and C. S. Ong, “Factorized Fourier Neural Operators,” 2021. arXiv:2111.13802. <https://arxiv.org/abs/2111.13802>